METRIC_DEPENDENCY_GRAPH

Helios — Metric Dependency Graph

Companion to: `models/semantic/semantic_layer.yaml` (the registry) · **Version:** v1.0 · **Date:** 2026-06-03

Purpose. This document is the human-readable map of how every Helios metric is *built from* every other metric. The semantic registry is the machine source of truth (`numerator` / `denominator` / `expr` references); this file renders those references as dependency trees, a consolidated DAG, and the decomposition paths the Diagnose / Decompose agents follow during root-cause analysis (RCA). It exists so that:

- A returning engineer (or a resuming agent) can see, at a glance, the *blast radius* of any metric — what breaks if a leaf measure is wrong, and what a ratio rolls up into.
- `stats-mcp.decompose_change` and the Diagnose hypothesis tree have a written, agreed drill order per headline metric.
- The algebraic **identities** that let RPS / revenue movement be attributed to conversion vs basket-size are stated once, authoritatively.

Scope & rules. Every node below is an *exact* `metric_name` from the PINNED REGISTRY — no synonyms, no invented names. Edges point from a metric to the metrics it directly depends on. **Leaves** are `count` / `sum` measures (named or `supporting`) that resolve to a single additive SQL expression over a physical grain. **Ratios** depend on exactly two metrics (numerator, denominator). **Derived** metrics depend on the metrics named in their `expr` (or, for window metrics, on themselves plus an `OVER()` frame). Cross-cluster references (e.g. `revenue_per_user` in cluster 01 referencing `users` in cluster 02) are legal and resolve in the final registry merge.

Legend for the trees:

- `name` — a metric_name. A trailing `(sum, fct_orders)` etc. shows `type` and `grain` for leaves.
- `→` "depends on". `x` (times) joins co-dependencies of a derived/identity expression.
- `[S]` marks a **supporting** measure (category: `supporting` ; defined inside the cluster that uses it).
- `[req: channel_group]` marks a metric the registry requires be sliced by that dimension.

---

1. Per-metric dependency trees

1.1 Revenue (cluster 01)

```
revenue (sum, fct_funnel)            → session_revenue                    [leaf]
gross_revenue (sum, fct_orders)      → gross_revenue                      [leaf]
net_revenue (sum, fct_orders)        → net_revenue                        [leaf]
orders (count, fct_orders)           → order_key (count_distinct)         [leaf]

aov (ratio, fct_orders)
  → gross_revenue                    (sum, fct_orders)                    [leaf]
  → orders                           (count, fct_orders)                  [leaf]
```

```
revenue_per_session (derived, fct_funnel)   expr SAFE_DIVIDE({revenue}, {sessions})
  → revenue                         (sum, fct_funnel)                       [leaf]
  → sessions                        (count, fct_funnel)                     [leaf]   (cluster 02)

revenue_per_user (derived, fct_funnel)      expr SAFE_DIVIDE({revenue}, {users})
  → revenue                         (sum, fct_funnel)                       [leaf]
  → users                           (count, fct_funnel)                     [leaf]   (cluster 02)
```

1.2 Traffic (cluster 02)

```
users (count, fct_funnel)            → user_pseudo_id (count_distinct)         [leaf]
new_users (count, fct_funnel)        → IF(is_new_user, user_pseudo_id, NULL)   [leaf]
returning_users (count, fct_funnel)  → IF(NOT is_new_user, user_pseudo_id,NULL)[leaf]
sessions (count, fct_funnel)         → session_key (count_distinct)            [leaf]
engaged_sessions (count, fct_sessions) → engaged_session (countif)             [leaf]

engagement_rate (ratio, fct_sessions)
  → engaged_sessions                (count, fct_sessions)                     [leaf]
  → sessions                        (count, fct_funnel)                       [leaf]
```

1.3 Funnel step counts (cluster 03)

```
view_item_sessions (count, fct_funnel)      → reached_view_item (countif)      [leaf]
add_to_cart_sessions (count, fct_funnel)    → reached_add_to_cart (countif)    [leaf]
begin_checkout_sessions (count, fct_funnel) → reached_begin_checkout (countif) [leaf]
purchasing_sessions (count, fct_funnel)     → reached_purchase (countif)       [leaf]
```

These four are **monotonic** by construction (max-downstream `reached_*` flags), so `sessions ≥ view_item_sessions ≥ add_to_cart_sessions ≥ begin_checkout_sessions ≥ purchasing_sessions`, which guarantees every step rate below lands in `[0, 1]`.

1.4 Conversion rates (cluster 03)

```
view_to_cart_rate (ratio, fct_funnel)
  → add_to_cart_sessions            (count, fct_funnel)                       [leaf]
  → view_item_sessions              (count, fct_funnel)                       [leaf]

cart_to_checkout_rate (ratio, fct_funnel)
  → begin_checkout_sessions         (count, fct_funnel)                       [leaf]
  → add_to_cart_sessions            (count, fct_funnel)                       [leaf]

checkout_to_purchase_rate (ratio, fct_funnel)
  → purchasing_sessions             (count, fct_funnel)                       [leaf]
  → begin_checkout_sessions         (count, fct_funnel)                       [leaf]

session_conversion_rate (ratio, fct_funnel)
  → purchasing_sessions             (count, fct_funnel)                       [leaf]
  → sessions                        (count, fct_funnel)                       [leaf]   (cluster 02)

user_conversion_rate (ratio, fct_funnel)
  → purchasing_users [S]            (count, fct_funnel)                       [leaf]
  → users                           (count, fct_funnel)                       [leaf]   (cluster 02)
```

```
purchasing_users [S] (count, fct_funnel)   → IF(reached_purchase, user_pseudo_id, NULL) (count_distinct)  [leaf]
```

The whole-funnel rate factors into the three step rates — see Identities.

1.5 Retention (cluster 04)

```
day_1_retention (ratio, fct_cohorts)
  → retained_users_d1 [S]           (count, fct_cohorts)                      [leaf]
  → cohort_size [S]                 (count, fct_cohorts)                      [leaf]

day_7_retention (ratio, fct_cohorts)
  → retained_users_d7 [S]           (count, fct_cohorts)                      [leaf]
  → cohort_size [S]                 (count, fct_cohorts)                      [leaf]

day_30_retention (ratio, fct_cohorts)
  → retained_users_d30 [S]          (count, fct_cohorts)                      [leaf]
  → cohort_size [S]                 (count, fct_cohorts)                      [leaf]

repeat_purchase_rate (ratio, fct_orders)
  → repeat_purchasers [S]           (count, fct_orders)                       [leaf]   (users with ≥2 orders)
  → purchasers [S]                  (count, fct_orders)                       [leaf]   (users with ≥1 order)

cohort_size [S]        (count, fct_cohorts)  → cohort acquisition-week user count       [leaf]
retained_users_d1 [S]  (count, fct_cohorts)  → users active at age d1                   [leaf]
retained_users_d7 [S]  (count, fct_cohorts)  → users active at age d7                   [leaf]
retained_users_d30 [S] (count, fct_cohorts)  → users active at age d30                  [leaf]
purchasers [S]         (count, fct_orders)   → distinct users with ≥1 order             [leaf]
repeat_purchasers [S]  (count, fct_orders)   → distinct users with ≥2 orders            [leaf]
```

Retention denominators are the **fixed** original cohort size (`cohort_size`), never the surviving population, so dN retention is monotonically non-increasing in N.

1.6 Acquisition (cluster 04)

```
channel_revenue (sum, fct_orders) [req: channel_group]
  → gross_revenue                   (the physical sql column, summed)         [leaf]

channel_conversion_rate (ratio, fct_funnel) [req: channel_group]
  → purchasing_sessions             (count, fct_funnel)                       [leaf]   (cluster 03)
  → sessions                        (count, fct_funnel)                       [leaf]   (cluster 02)

cac_proxy (derived, fct_funnel)   expr SAFE_DIVIDE({paid_sessions}, {new_purchasers})
  → paid_sessions [S]               (count, fct_funnel)                       [leaf]
  → new_purchasers [S]              (count, fct_funnel)                       [leaf]

traffic_share (derived/window, fct_funnel)  sql SAFE_DIVIDE(sessions, SUM(sessions) OVER ())
  → sessions                        (count, fct_funnel)                       [leaf]   (cluster 02)
  → SUM(sessions) OVER ()           (window frame over the same leaf)

paid_sessions [S] (count, fct_funnel)
  → countif channel_group IN ('Paid Search','Paid Social','Display')          [leaf]
new_purchasers [S] (count, fct_funnel)
  → count_distinct new users who purchased                                    [leaf]
```

`cac_proxy` **honesty note.** This is *not* true CAC. True CAC = `marketing_spend / new_customers`, but the GA4 obfuscated sample has **no cost data**. `cac_proxy` is a volume-efficiency proxy (paid sessions per new customer). It must never be read as a dollar acquisition cost; a real CAC requires an external cost table (deferred per the Bible roadmap). `traffic_share` is the mix weight `w_i` consumed by the mix-vs-rate decomposition, not a performance metric.

1.7 Product (cluster 05, item-level)

```
product_revenue (sum, fct_order_items)   dims item_name, item_category, item_brand
  → item_revenue_in_usd                  (the physical sql column, summed)      [leaf]

product_conversion_rate (ratio, fct_order_items)
  → item_purchases [S]                   (count, fct_order_items)               [leaf]
  → item_views [S]                       (count, fct_order_items)               [leaf]

product_view_rate (ratio, fct_funnel)
  → item_view_sessions [S]               (count, fct_funnel)                    [leaf]
  → view_item_sessions                   (count, fct_funnel)                    [leaf]   (cluster 03)

product_attach_rate (ratio, fct_order_items)
  → orders_with_item [S]                 (count, fct_order_items)               [leaf]
  → orders                               (count, fct_orders)                    [leaf]   (cluster 01, NAMED)

item_views [S]         (count, fct_order_items) → item-level view_item occurrences      [leaf]
item_purchases [S]     (count, fct_order_items) → item-level purchase occurrences       [leaf]
item_view_sessions [S] (count, fct_funnel)      → distinct sessions viewing the item    [leaf]
orders_with_item [S]   (count, fct_order_items) → distinct orders containing the item   [leaf]
```

Note the **cross-grain** edge in `product_attach_rate`: its denominator `orders` is the NAMED metric on `fct_orders` (cluster 01), while its numerator `orders_with_item` is item-level on `fct_order_items`. The registry resolves both; the resolver must reconcile the order grain across the two facts.

---

2. Consolidated DAG: leaves -> ratios -> derived

Three layers. Layer 0 = additive `count` / `sum` leaves (named + supporting). Layer 1 = `ratio` metrics (each consumes exactly two Layer-0 nodes). Layer 2 = `derived` / window metrics. No edge points upward; the graph is acyclic.

```
LAYER 0 — LEAVES (count / sum measures; the only nodes touching physical columns)
  fct_funnel:       revenue  sessions  users  new_users  returning_users
                    view_item_sessions  add_to_cart_sessions
                    begin_checkout_sessions  purchasing_sessions
                    purchasing_users[S]  paid_sessions[S]  new_purchasers[S]
                    item_view_sessions[S]
  fct_sessions:     engaged_sessions
  fct_orders:       gross_revenue  net_revenue  orders
                    purchasers[S]  repeat_purchasers[S]
  fct_order_items:  product_revenue  item_views[S]  item_purchases[S]  orders_with_item[S]
  fct_cohorts:      cohort_size[S]  retained_users_d1[S]
                    retained_users_d7[S]  retained_users_d30[S]
        |
        v
LAYER 1 — RATIOS (numerator / denominator, both Layer-0 leaves)
```

```
  engagement_rate            = engaged_sessions        / sessions
  view_to_cart_rate          = add_to_cart_sessions    / view_item_sessions
  cart_to_checkout_rate      = begin_checkout_sessions / add_to_cart_sessions
  checkout_to_purchase_rate  = purchasing_sessions     / begin_checkout_sessions
  session_conversion_rate    = purchasing_sessions     / sessions
  user_conversion_rate       = purchasing_users[S]     / users
  channel_conversion_rate    = purchasing_sessions     / sessions      [req: channel_group]
  aov                        = gross_revenue           / orders
  day_1_retention            = retained_users_d1[S]    / cohort_size[S]
  day_7_retention            = retained_users_d7[S]    / cohort_size[S]
  day_30_retention           = retained_users_d30[S]   / cohort_size[S]
  repeat_purchase_rate       = repeat_purchasers[S]    / purchasers[S]
  product_conversion_rate    = item_purchases[S]       / item_views[S]
  product_view_rate          = item_view_sessions[S]   / view_item_sessions
  product_attach_rate        = orders_with_item[S]     / orders
        |
        v
LAYER 2 — DERIVED / WINDOW (expr over metrics; may reference Layer 0 and/or Layer 1)
  revenue_per_session  = SAFE_DIVIDE({revenue}, {sessions})           (= session_conversion_rate x aov; see Identitie
  revenue_per_user     = SAFE_DIVIDE({revenue}, {users})
  cac_proxy            = SAFE_DIVIDE({paid_sessions[S]}, {new_purchasers[S]})   (NOT true CAC)
  traffic_share        = SAFE_DIVIDE(sessions, SUM(sessions) OVER ())           (mix weight w_i)

PURE LEAVES (named, no dependents downstream of themselves):
  channel_revenue (= SUM gross_revenue [req: channel_group]),  product_revenue,
  net_revenue,  new_users,  returning_users
```

Reading the blast radius: a defect in the leaf `purchasing_sessions` propagates to **five** downstream metrics (`checkout_to_purchase_rate`, `session_conversion_rate`, `channel_conversion_rate`, plus indirectly to `revenue_per_session` via the conversion identity). A defect in `sessions` is even broader (`engagement_rate`, `session_conversion_rate`, `channel_conversion_rate`, `revenue_per_session`, `traffic_share`). These two are the highest-leverage leaves to test.

---

3. Decomposition paths for RCA

For each headline metric, the recommended **drill order** and the **mix-vs-rate** read. The pattern is always: detect (`Monitor.detect_anomaly`) -> decompose along the canonical split with `stats-mcp.decompose_change` (`mix_effect + rate_effect + interaction`) -> if the move is **mix-dominant** it is a composition artifact (acquisition / targeting action); if **rate-dominant** it is real in-segment behavior (funnel / UX / pricing fix); drill the segment with the largest **rate** effect. All rates are recomputed as `SUM(num)/SUM(den)` after grouping (Simpson's-paradox defense).

Headline metric	Primary split (decompose_change)	Drill order	Mix-dominant =>	Rate-dominant =>
`revenue`	`channel_group` x `device_category`	revenue -> orders x aov -> session_conversion_rate x aov -> funnel step rates	traffic-mix shift moved revenue; act on acquisition / channel allocation	conversion or basket-size moved; drill the conversion identity then `aov`
`session_conversion_rate`	`channel_group` x `device_category`	session_conversion_rate -> view_to_cart_rate / cart_to_checkout_rate / checkout_to_purchase_rate; then landing_page, is_new_user, source/medium	low-converting channel/device grew its share; acquisition / spend reallocation	a real step regressed in-segment; fix that funnel step (UX / latency / form)
`revenue_per_session`	`channel_group` x `device_category`	split RPS = session_conversion_rate x aov; whichever factor moved, recurse into its own tree	mix of high-RPS segments shrank; rebalance acquisition	conversion-driven (-> step rates) or basket-driven (-> aov -> item mix)
`aov`	`item_category` x `channel_group`	aov -> gross_revenue / orders; then item_brand, item_name, product mix	basket mix shifted toward cheaper items/categories; merchandising / bundling	per-item price or units-per-order changed in-segment; pricing / promo review
`checkout_to_purchase_rate`	`device_category` x `operating_system` (then `browser`, `landing_page`)	checkout_to_purchase_rate -> begin_checkout_sessions vs purchasing_sessions; isolate device/OS/browser; correlate with launch_calendar	a high-friction device/OS grew its checkout share	payment / checkout regression on a specific device-OS-browser; ship a fix + experiment

Headline metric	Primary split (decompose_change)	Drill order	Mix-dominant =>	Rate-dominant =>
`channel_conversion_rate`	`channel_group` (required) x `device_category`	channel_conversion_rate per channel -> compare vs `traffic_share` (w_i) to separate mix from rate; drill the worst channel's funnel	a channel's session share grew faster than its rate (composition)	that channel's in-funnel rate genuinely dropped; channel-specific funnel fix

Cross-checks the Critic re-runs. (1) Does the decomposition reconcile: `mix_effect + rate_effect + interaction == ΔR` within tolerance? (2) Is a "rate" finding really a confounded mix at a finer grain (re-split one level deeper)? (3) Does the dollar impact tie to `revenue` / `channel_revenue` via the identities below? (4) For any `channel_*` finding, is the apparent rate move explained by `traffic_share` shifting (Simpson's paradox)?

---

4. Quick reference: where each metric is consumed

Metric	Layer	Drives (downstream)	Typical agents
`sessions`	leaf	engagement_rate, session_conversion_rate, channel_conversion_rate, revenue_per_session, traffic_share	Monitor, Decompose, Diagnose
`purchasing_sessions`	leaf	checkout_to_purchase_rate, session_conversion_rate, channel_conversion_rate	Monitor, Decompose, Diagnose, Critic
`gross_revenue` / `orders`	leaf	aov, channel_revenue	Monitor, Narrator, Prescribe
`revenue`	leaf	revenue_per_session, revenue_per_user	Monitor, Narrator, Critic
`session_conversion_rate`	ratio	revenue_per_session (via identity)	Decompose, Diagnose, Prescribe, Critic, Narrator
`aov`	ratio	revenue (via identity), revenue_per_session (via identity)	Decompose, Prescribe, Narrator
step rates	ratio	session_conversion_rate (via identity)	Decompose, Diagnose, Critic
`revenue_per_session`	derived	headline RPS	Monitor, Decompose, Narrator
`traffic_share`	window	mix weight `w_i` for decompose_change	Decompose, Diagnose, Critic

Metric	Layer	Drives (downstream)	Typical agents
`cac_proxy`	derived	acquisition-efficiency read (proxy only)	Diagnose, Prescribe

5. Identities

These exact algebraic identities hold by construction and are what let the agents attribute a dollar/rate movement to a specific factor. They are stated in terms of registry `metric_name`s only.

```
(I1)  revenue_per_session  =  session_conversion_rate  x  aov
      SAFE_DIVIDE(revenue, sessions)
        =  SAFE_DIVIDE(purchasing_sessions, sessions)  x  SAFE_DIVIDE(gross_revenue, orders)
      Reads RPS movement as conversion-driven vs basket-driven.
      (Holds when session-attributed revenue ties to order revenue; reconcile within 0.5%.)

(I2)  revenue  =  orders  x  aov
      SUM(gross_revenue)  =  orders  x  SAFE_DIVIDE(gross_revenue, orders)
      Splits a revenue move into order-volume vs basket-size.

(I3)  session_conversion_rate  =  view_to_cart_rate  x  cart_to_checkout_rate  x  checkout_to_purchase_rate
      = (add_to_cart_sessions / view_item_sessions)
        x (begin_checkout_sessions / add_to_cart_sessions)
        x (purchasing_sessions / begin_checkout_sessions)
      The intermediate counts telescope to (purchasing_sessions / view_item_sessions).
      NOTE: this is the view-anchored funnel product; session_conversion_rate's own
      denominator is `sessions` (all sessions), so the full chain from sessions is:
        session_conversion_rate
          = (view_item_sessions / sessions) x view_to_cart_rate
            x cart_to_checkout_rate x checkout_to_purchase_rate.
      The three named step rates account for the view_item → purchase portion; the
      sessions → view_item entry stage is the remaining factor.
```

Composite chain (how the agents thread the identities for a top-down RCA):

```
revenue
  = orders x aov                                  ............ (I2)
  ~ (sessions x session_conversion_rate) x aov    (orders ~ purchasing_sessions at session grain)
  = sessions x (session_conversion_rate x aov)
  = sessions x revenue_per_session                ............ (I1)

where session_conversion_rate
  = (view_item_sessions / sessions)
    x view_to_cart_rate x cart_to_checkout_rate x checkout_to_purchase_rate    ... (I3)
```

So a `revenue` anomaly is walked down to exactly one of: **traffic volume** (`sessions`), a specific **funnel step rate** (`view_to_cart_rate` / `cart_to_checkout_rate` / `checkout_to_purchase_rate`), or **basket size** (`aov`) — and `traffic_share` separates whether any of those moved because of **mix** (composition across `channel_group` / `device_category`) rather than a genuine in-segment **rate** change. That walk is exactly the drill order in section 3.